

Bản sơ về ứng dụng của chiến lược nhị phân

Yang Jun

2005

Nguồn gốc: Bản sơ về ứng dụng của chiến lược nhị phân

IOI China candidate team papers / 2005 / Yang Jun.doc

Tóm tắt

Bài viết tập trung thảo luận ba kiểu bài toán nhị phân khác nhau, với mục đích giúp người đọc hiểu sâu hơn về phương pháp nhị phân. Đối tượng được xét đi từ dãy có thứ tự thông thường, đến dãy có thứ tự đã suy biến, rồi cuối cùng là dãy vô thứ tự. Trên thực tế, chúng cũng đại diện cho ba cách ứng dụng khác nhau của chiến lược nhị phân.

Từ khóa: nhị phân; thứ tự; ứng dụng.

1 Mở đầu

“Nhị phân” hẳn là một từ không còn xa lạ với mọi người. Nhị phân là một quy tắc sàng lọc, bắt nguồn từ một ý tưởng rất đơn giản: trong trường hợp xấu nhất, loại bỏ càng nhiều càng tốt để tìm được mục tiêu nhanh nhất có thể.

Hiệu quả của thuật toán nhị phân đến từ việc nó tận dụng đầy đủ thông tin, cố gắng loại bỏ phần dư thừa và giảm những phép tính không cần thiết, từ đó tối đa hóa hiệu suất của thuật toán. Thực ra, nhiều bài toán nhị phân có thể được chứng minh bằng cây quyết định hoặc một số định lý khác rằng chúng đạt tới cận dưới độ phức tạp của bài toán.

Dù bản thân tư tưởng nhị phân rất đơn giản, khả năng mở rộng và phạm vi ứng dụng rộng của nó có lẽ vẫn vượt ngoài dự liệu ban đầu của chúng ta. Trong các kỳ thi những năm gần đây, ta cũng thấy không ít ví dụ khiến người đọc bất ngờ về cách dùng tư tưởng nhị phân. Dưới đây là ba kiểu ứng dụng khác nhau của tư tưởng nhị phân mà tác giả tổng kết lại, hy vọng có ích cho người đọc.

2 Dạng một: tìm kiếm nhị phân trên dãy có thứ tự thông thường

Trước hết cần nói rõ: dãy có thứ tự ở đây không chỉ giới hạn ở dãy đã được sắp từ nhỏ đến lớn hoặc từ lớn đến nhỏ như ta thường nói. Nó chỉ gồm hai ý: thứ nhất, đó là một dãy một chiều; thứ hai, dãy ấy có thứ tự, tức là hai phần tử bất kỳ trong dãy đều có thể so sánh được. Nói cách khác, nó có quan hệ thứ tự toàn phần quen thuộc.

Tuy mọi người đều rất quen với tìm kiếm nhị phân, ta vẫn nhắc lại ngắn gọn quá trình cài đặt tổng quát:

1. Xác định phạm vi chứa phần tử cần tìm.
2. Chọn một phần tử trong phạm vi đó làm mốc.

3. So sánh khóa của phần tử cần tìm với khóa của phần tử mốc, rồi xác định phạm vi mới, chính xác hơn, của phần tử cần tìm.
4. Nếu phạm vi mới đủ chính xác thì xuất kết quả; nếu không, quay lại bước 2.

2.1 Bài toán thống kê thứ tự

Mô tả bài toán. Cho một tập S gồm n số đôi một khác nhau. Hãy tìm phần tử nhỏ thứ i trong tập đó.

Phân tích. Hẳn mọi người đều quen thuộc với bài toán này. Hãy nhắc lại cách áp dụng tìm kiếm nhị phân vào bài toán:

1. Xác định rằng phần tử cần tìm nằm trong S .
2. Chọn ngẫu nhiên một trong n phần tử, ký hiệu là x , và dùng x làm mốc.
3. Gọi p là số phần tử trong S nhỏ hơn x .
4. Nếu $i < p$, phần tử cần tìm nhỏ hơn x , nên ta thu hẹp phạm vi về các phần tử của S nhỏ hơn x và tìm phần tử thứ i trong phạm vi đó.
5. Nếu $i > p$, phần tử cần tìm lớn hơn x , nên ta thu hẹp phạm vi về các phần tử của S lớn hơn x và tìm phần tử thứ $i - p - 1$ trong phạm vi đó.
6. Nếu $i = p$, phần tử nhỏ thứ i chính là x .
7. Nếu đã tìm được x thì xuất kết quả; nếu không, quay lại bước 2.

Vì x được chọn ngẫu nhiên, một phân tích xác suất đơn giản cho thấy độ phức tạp kỳ vọng của thuật toán là $\mathcal{O}(n)$.

Tiểu kết. Ví dụ này nhằm nhắc mọi người hai điểm.

Thứ nhất, đừng mặc nhiên cho rằng tìm kiếm nhị phân nhất định phải liên quan đến $\log n$. Bước 3 trong thuật toán, tức bước “chia” theo cách nói thông thường, không đòi hỏi lần nào cũng phải thực hiện trong thời gian $\mathcal{O}(1)$. Việc chia có thể dựa trên xử lý hiệu quả đối với dãy, chẳng hạn như phép phân hoạch tập hợp tương tự quicksort trong ví dụ trên.

Thứ hai, phép “chia” của thuật toán nhị phân không bắt buộc lần nào cũng phải cân bằng. Đôi khi điều đó rất khó làm. Chỉ cần không lần nào cũng lệch hẳn, ta đã có thể thu được một thuật toán hiệu quả; điều này cũng tạo cơ hội cho việc dùng thuật toán ngẫu nhiên.

Trong những năm gần đây, sự xuất hiện của các bài tương tác cũng làm tìm kiếm nhị phân thêm sinh động. Khá nhiều bài tìm kiếm nhị phân được đưa ra dưới dạng bài tương tác. Chẳng hạn, ví dụ trên chỉ cần biến đổi một chút là thành một bài tương tác độ khó trung bình ở IOI 2000. Hai bài gần như cùng khuôn: bạn hỏi phần tử nhỏ thứ i , còn bài kia hỏi phần tử nhỏ thứ $(N + 1)/2$; cách giải cũng bất chước tương tự. Bạn chọn ngẫu nhiên x rồi chia thành hai đoạn theo quan hệ lớn nhỏ với x ; bài kia chọn ngẫu nhiên x, y rồi chia thành ba đoạn theo quan hệ lớn nhỏ với x, y . Độ phức tạp kỳ vọng của bạn là $\mathcal{O}(n)$, còn số câu hỏi kỳ vọng của bài kia là $\mathcal{O}(N)$. Chi tiết cụ thể không được trình bày ở đây; bạn đọc quan tâm có thể tham khảo báo cáo lời giải của các anh chị đi trước.¹

¹Bài “Báo cáo lời giải độ khó trung bình” của Gao Yue, tuyển tập luận văn đội tuyển huấn luyện 2000–2002.

3 Dạng hai: liệt kê nhị phân trên dãy có thứ tự suy biến

Chiến lược nhị phân không phải lúc nào cũng được áp dụng trên một dãy có thứ tự hiển nhiên như trên. Nó có thể dựa vào một mối liên hệ tiềm ẩn nào đó của bài toán, rồi áp dụng lên một số dãy có thứ tự suy biến ngầm định. So với tìm kiếm nhị phân đã giới thiệu ở trên, khác biệt lớn nhất là ở đây, khi quyết định đệ quy sang phần nào, phép nhị phân không dùng phép so sánh trực tiếp.

3.1 Giải quần vợt chuyên nghiệp BTP

Mô tả bài toán. Bài này đến từ USACO contest tháng 12 năm 2002. Có N con bò, trong đó $N = 2^K$, đều là cao thủ quần vợt. Mỗi con bò có một hạng BTP, đúng bằng một số từ 1 đến N . Tuần sau sẽ có một giải đấu loại trực tiếp: N con bò được chia thành $N/2$ cặp, mỗi cặp đấu để chọn ra $N/2$ người thắng; rồi $N/2$ người thắng lại được chia thành $N/4$ cặp, và cứ thế cho đến khi còn lại một con bò vô địch.

Thi đấu vừa xét thực lực vừa xét vận may. Nếu hạng BTP của hai con bò chênh lệch hơn số nguyên cho trước K , con bò có hạng tốt hơn luôn thắng con có hạng kém hơn; ngược lại, hai bên đều có khả năng thắng. Khán giả muốn biết con bò có hạng kém nhất trong số tất cả các con bò có thể trở thành vô địch là con nào, đồng thời yêu cầu liệt kê một cách xếp lịch thi đấu khiến con bò đó thắng. Giới hạn là $N \leq 65536$.

Phân tích. Vì N rất lớn, ta đoán rằng có thể giải bằng tham lam.

Ta muốn các tuyển thủ hạng tốt hơn luôn “bắt ngờ” thua các tuyển thủ hạng kém hơn. Vì vậy ta cho 1 thua $K + 1$, 2 thua $K + 2$, v.v. Trong mỗi ván của mỗi vòng, luôn chọn tuyển thủ có hạng tốt nhất trong số chưa thi đấu, rồi cho thua một tuyển thủ có hạng kém nhất có thể.

Nhưng rất dễ tìm phản ví dụ. Khi $N = 8, K = 2$, cách tham lam trên cho kết quả là 4 ở sơ đồ BTP-1, trong đó X (Y nghĩa là X thắng Y):

```
3(1  4(2  7(5  8(6
4(3      8(7
4(8
```

Trong khi đó, nghiệm tối ưu là 6 ở sơ đồ BTP-2:

```
6(7  5(3  4(8  2(1
6(5      4(2
6(4
```

Nguyên nhân là ta không biết nghiệm tối ưu bằng bao nhiêu, nên đang tham lam một cách mù quáng. Trên thực tế, nghiệm tối ưu 6 đã bị ta loại ngay ở vòng đầu, vì thế tất nhiên không thể thu được nghiệm tối ưu.

Muốn biết nghiệm tối ưu? Hãy liệt kê. Đồng thời, xét một kết luận rất hiển nhiên: nếu tuyển thủ hạng X có thể thắng chung cuộc, thì mọi tuyển thủ có hạng tốt hơn X cũng có thể thắng chung cuộc.

Dù kết luận có vẻ hiển nhiên, để chứng minh nó vẫn cần suy nghĩ một chút. Giả sử tuyển thủ hạng X thắng chung cuộc. Ta sửa cục bộ cách xếp lịch ấy để xây dựng một cách xếp lịch mới khiến một tuyển thủ bất kỳ hạng $Y < X$ thắng. Trong lịch mà X thắng chung cuộc, giả sử Y bị Z đánh bại. Nếu $Z \neq X$, do $X > Y$, ta biết Z cũng có thể đánh bại X , và trong cùng vòng ấy cũng như các vòng sau, mọi đối thủ mà X đánh bại đều có thể bị Y đánh bại. Vì vậy ta chỉ cần cho Z đánh bại X ở vòng này, rồi đổi mọi lần xuất hiện của X trong các trận sau thành Y . Nếu $Z = X$, do $X > Y$, ta cho Y đánh bại X , rồi cũng đổi mọi trận sau của X thành Y ; cuối cùng người thắng vẫn là Y .

Nhờ đó, ta có thể liệt kê nhị phân giá trị X của người thắng chung cuộc, nâng cao hiệu quả thuật toán đáng kể.

Bây giờ vấn đề là: nếu đã biết người thắng chung cuộc là X , ta có thể nhanh chóng xây dựng một cách xếp lịch, hoặc chứng minh rằng không tồn tại, hay không? Có. Ta vẫn dùng tham lam, nhưng vì đã biết ai thắng cuối cùng nên ta làm ngược. Ở mỗi vòng, ta để các tuyển thủ đã có mặt đánh bại tuyển thủ có hạng tốt nhất trong số chưa xuất hiện. Lịch đấu sinh ra bởi cách này chính là kiểu được mô tả trong BTP-2. Cách tìm tuyển thủ chưa xuất hiện có hạng tốt nhất có thể dùng cây nhị phân tìm kiếm tĩnh; phần này không trình bày thêm, bạn đọc có thể tự suy nghĩ.

Có thể chứng minh cách tham lam này đúng; phương pháp chứng minh tương tự chứng minh phía trên nên không lặp lại. Toàn bộ bài toán có thể hoàn thành trong thời gian $\mathcal{O}(N \log_2 N)$.

Tiểu kết. Đối với dạng bài cần liệt kê nhị phân này, bản chất của thuật toán là tìm kiếm nhị phân trên một dãy có thứ tự suy biến ngầm định, chỉ có hai giá trị 0 và 1. Trong ví dụ trên, khi $X < \text{Ans}$ thì $A[X] = 0$, còn khi $X \geq \text{Ans}$ thì $A[X] = 1$. Điều ta cần tìm chính là điểm phân giới giữa hai loại giá trị ấy. Có điểm phân giới thì có giá trị tối ưu, và nhờ đó có lời giải cho bài toán ban đầu.

3.2 Phân phát huy chương

Mô tả bài toán. Bài này đến từ ACM/ICPC CERC 2004. Có một bức tường hình vòng tròn; trên tường có một số tháp, mỗi tháp có một lính gác. Bây giờ cần phát cho mỗi lính gác một số huy chương. Lính gác thứ i cần $P[i]$ huy chương. Các huy chương của cùng một lính gác phải thuộc các loại khác nhau, và hai lính gác kề nhau không được có bất kỳ huy chương nào cùng loại. Hỏi ít nhất cần chuẩn bị bao nhiêu loại huy chương khác nhau. Giới hạn là $2 \leq n \leq 10000$ và $1 \leq P[i] \leq 100000$.

Phân tích. Nếu bức tường không phải hình vòng tròn, ta rất dễ giải bằng tham lam: mỗi lần phát cho lính gác nhiều nhất có thể từ các huy chương đã chuẩn bị; nếu không đủ thì chuẩn bị thêm một số loại để thỏa mãn nhu cầu. Nhưng bây giờ bức tường là vòng tròn, lính gác cuối cùng và lính gác đầu tiên cũng không được trùng loại huy chương, và đây là điểm khó của bài toán.

Vì vậy ta thử đưa điểm khó này vào trạng thái và dùng quy hoạch động² để giải, vì quy hoạch động là một phương pháp thường dùng cho các bài toán tối ưu.

Gọi $a[i, j]$ là số loại huy chương tối thiểu cần thêm, ngoài $P[1]$ loại huy chương đã cấp cho lính gác đầu tiên, sau khi đã sắp xếp xong i lính gác đầu và lính gác thứ i có j huy chương cùng loại với lính gác đầu tiên.

Giả sử lính gác thứ $i - 1$ có k huy chương cùng loại với lính gác đầu tiên. Vì k huy chương ấy và j huy chương của lính gác thứ i phải đôi một khác loại, ta có ngay $j + k \leq P[1]$. Mặt khác, $P[i - 1] - k$ huy chương còn lại của lính gác thứ $i - 1$ cũng phải khác với $P[i] - j$ huy chương còn lại của lính gác thứ i . Gọi số loại huy chương cần tăng thêm là X , ta có

$$a[i - 1, k] + X \geq (P[i] - j) + (P[i - 1] - k),$$

suy ra

$$X \geq (P[i] - j) + (P[i - 1] - k) - a[i - 1, k].$$

Do đó có thể viết chuyển trạng thái dưới dạng

$$a[i, j] = \min_{j+k \leq P[1]} \max_k \{a[i - 1, k], (P[i] - j) + (P[i - 1] - k)\}.$$

Làm trực tiếp như vậy có độ phức tạp tới $\mathcal{O}(nP_{\max}^2)$. Dù có tối ưu, ta cũng khó thu được kết quả thỏa đáng, nên cần tìm lối khác.

²Nói chính xác hơn ở đây là truy hồi.

Xét tính chất sau: nếu với $P[1] + X$ loại huy chương tồn tại một phương án phân phát thỏa yêu cầu, thì với $P[1] + X + 1$ loại cũng chắc chắn có phương án khả thi; cùng lắm ta không dùng một loại nào đó. Tính chất này rất quan trọng, vì nhờ nó, ta có thể liệt kê nhị phân X rồi lần lượt kiểm tra tính khả thi để thu được kết quả.

Vậy bài toán ban đầu được chuyển thành: nếu đã biết tổng cộng có $P[1] + X$ loại huy chương, liệu ta có thể nhanh chóng kiểm tra có tồn tại một cách phân phát thỏa yêu cầu hay không? Câu trả lời là có.

Phương pháp vẫn là quy hoạch động, và cách biểu diễn trạng thái cũng tương tự. Gọi $a[i, j]$ cho biết, khi tổng số loại huy chương là $P[1] + X$, có thể sắp xếp xong i lính gác đầu và làm cho lính gác thứ i có j huy chương cùng loại với lính gác đầu tiên hay không. Khi đó chuyển trạng thái trở thành:

$$a[i, j] = \text{true} \iff \exists k : \begin{cases} a[i-1, k] = \text{true}, \\ j + k \leq P[1], \\ (P[i] - j) + (P[i-1] - k) \leq X. \end{cases}$$

Nhìn bề ngoài, chuyển trạng thái vẫn khá rườm rà, vì k có rất nhiều ràng buộc. Nhưng quan sát kỹ, hai điều kiện thứ hai và thứ ba hợp lại thành

$$P[i-1] + P[i] - X - j \leq k \leq P[1] - j.$$

Với i cố định, miền giá trị của k trên trục số là một đoạn có độ dài xác định, và vị trí của đoạn do j quyết định. Ràng buộc thứ nhất ban đầu chỉ là giới hạn thêm phạm vi cho đoạn này, loại bỏ phần dư thừa vô nghĩa.

Trạng thái ban đầu chỉ có $a[1, 0] = \text{true}$, các trạng thái còn lại đều là false. Bằng quy nạp, rất dễ chứng minh rằng với mọi i , tập các j sao cho $a[i, j]$ là true nhất định là một đoạn liên tiếp. Vì vậy, với mỗi i , tất cả các trạng thái $a[i, j]$ có thể được biểu diễn chỉ bằng hai đầu mút $a[i].x1$ và $a[i].x2$.

Để $a[i, j] = \text{true}$, khi và chỉ khi miền giá trị của k giao với đoạn $[a[i-1].x1, a[i-1].x2]$, tức là thỏa

$$P[1] - j \geq a[i-1].x1 \quad \text{và} \quad P[i-1] + P[i] - X - j \leq a[i-1].x2.$$

Nói cách khác,

$$P[i-1] + P[i] - X - a[i-1].x2 \leq j \leq P[1] - a[i-1].x1.$$

Từ đó, phương trình chuyển trạng thái trở thành

$$a[i].x1 = \max(0, P[i-1] + P[i] - X - a[i-1].x2),$$

$$a[i].x2 = \min(P[i], P[1] - a[i-1].x1).$$

Trạng thái ban đầu là $a[1].x1 = a[1].x2 = P[1]$. Số trạng thái là $\mathcal{O}(n)$, mỗi chuyển trạng thái tốn $\mathcal{O}(1)$, có thể nói là rất hiệu quả.

Với phương pháp này, dựa trên phân tích trước đó, ta liệt kê nhị phân X và dùng cách kiểm tra hiệu quả nói trên, từ đó giải toàn bộ bài toán trong thời gian $\mathcal{O}(n \log P_{\max})$.

Tiểu kết. Bài toán trên nhìn qua tưởng như điểm khó vẫn là quy hoạch động, còn liệt kê nhị phân chỉ là một công cụ phụ. Nhưng trên thực tế, việc liệt kê trước X đã làm phương trình quy hoạch động đơn giản đi rất nhiều, nhờ đó bài toán mới được giải quyết. Những bài tối ưu dùng tư tưởng liệt kê nhị phân như thế gần đây khá thịnh hành, và rất dễ dẫn dụ thí sinh trực tiếp dùng quy hoạch động hoặc tham lam rồi đi vào ngõ cụt.

Vì vậy, về sau khi xét các bài toán tối ưu, ta nên chú ý xem bài toán có ẩn chứa một dãy 01 có thứ tự hay không, và liệu có thể dùng liệt kê nhị phân để chuyển bài toán tối ưu thành bài toán khả thi hay không. Hãy ghi nhớ: lùi một bước, trời biển mở rộng.

4 Dạng ba: tìm kiếm nhị phân trên dãy vô thứ tự

Nếu bạn cho rằng chỉ trong dãy có thứ tự mới có thể dùng nhị phân, thì bạn đã sai lớn rồi; dãy vô thứ tự vẫn có thể được tìm kiếm bằng nhị phân. Hãy xem bài tương tác sau.

4.1 Chuyển đi của người bán hàng

Mô tả bài toán. Bài này đến từ JSOI. Là một người bán hàng, Mirko phải đi máy bay thăm N thành phố, mỗi thành phố đúng một lần. Giữa mỗi cặp thành phố có đúng một đường bay; máy bay luôn cất cánh vào đúng giờ và bay mất một giờ. Mỗi tuyến bay luôn có máy bay bay qua lại liên tục giữa hai thành phố: nếu có một máy bay lúc 5 giờ cất cánh từ thành phố A đến thành phố B , thì 6 giờ nó đến nơi, lập tức lại cất cánh, và 7 giờ quay về A , v.v.

Để bảo đảm hiệu quả, Mirko muốn sắp N thành phố thành một dãy $A_1, A_2, \dots, A_N$ sao cho với mỗi i ($1 < i < n$), sau khi tới thành phố A_i , Mirko có thể quảng cáo một giờ rồi lập tức rời A_i để đi tới A_{i+1} .

Đáng tiếc là Mirko không có lịch bay, nên anh phải gọi điện hỏi hãng hàng không. Mỗi cuộc gọi, anh có thể hỏi tuyến giữa hai thành phố A, B vào đúng 12 giờ trưa bay từ A sang B hay từ B sang A . Vì Mirko không muốn tốn quá nhiều tiền điện thoại, hãy giúp anh xác định một lộ trình bằng ít cuộc gọi nhất có thể. Chú ý rằng thư viện tương tác có thể điều chỉnh tuyến theo các câu hỏi của bạn để làm số cuộc gọi của bạn lớn nhất.

Phân tích. Bài toán khá dài, trước hết hãy phân tích xem rốt cuộc nó yêu cầu gì.

Giả sử 12 giờ trưa ta bay từ A_1 tới A_2 , 1 giờ đến nơi, rồi quảng cáo một giờ, 2 giờ rời A_2 để tới A_3 , v.v. Rõ ràng, ta luôn rời một thành phố để tới thành phố khác vào các giờ chẵn.

Vì thời gian bay khứ hồi đúng bằng 2 giờ và ở giữa không có thời gian nghỉ, nên nếu chuyển 12 giờ bay từ A tới B , thì vào mọi giờ chẵn, chuyển bay cũng nhất định bay từ A tới B .

Nhờ đó, ta có thể chuyển bài toán ban đầu thành bài toán tìm một đường Hamilton trong một tournament,³ và mục tiêu là dùng ít câu hỏi nhất có thể. Lưu ý đây là một phép chuyển hóa, không phải tương đương hoàn toàn.

Để tránh hỏi một cách mù quáng, ta thử dùng phương pháp tăng dần để mở rộng dãy. Trước hết hỏi tùy ý hai thành phố; cho tiện, chọn thành phố 1 và thành phố 2. Không mất tính tổng quát, giả sử có cung từ 1 đến 2, ta thu được một đường độ dài 1 là 1(2).

Giả sử ta đã thiết kế được một đường gồm i thành phố đầu, độ dài $i - 1$: $A_1(A_2(\dots(A_i$. Ta muốn thêm thành phố $i + 1$ vào để thành đường độ dài i .

Trước hết hỏi có cung từ $i + 1$ tới A_1 hay không. Nếu có, ta đổi đường thành $i + 1(A_1(A_2(\dots(A_i$. Nếu không, ta hỏi tiếp có cung từ A_i tới $i + 1$ hay không. Nếu có, ta đổi đường thành $A_1(A_2(\dots(A_i(i + 1$. Nếu vẫn không, điều đó nghĩa là không thể chèn ở hai đầu, nên $i + 1$ chỉ có thể chèn vào giữa.

Chú ý rằng cạnh giữa $i + 1$ và A_1 khi ấy có chiều từ A_1 tới $i + 1$. Nếu có cung từ $i + 1$ tới A_2 , ta có thể chèn $i + 1$ giữa A_1 và A_2 ; nếu $i + 1$ vẫn không có cung tới A_2 , tức là có cung từ A_2 tới $i + 1$, ta hỏi tiếp A_3 , v.v. Liệu cách hỏi này có thể đi tới cuối cùng mà vẫn không chèn được $i + 1$ hay không? Không, vì $i + 1$ có cung tới A_i .

Như vậy ta có một phương pháp hỏi, nhưng trong trường hợp xấu nhất, tổng số câu hỏi có thể là $\mathcal{O}(N^2)$.

Với mỗi điểm A_k ($1 \leq k \leq i$), hoặc A_k có cung tới $i + 1$, hoặc $i + 1$ có cung tới A_k . Hơn nữa A_1 có cung tới $i + 1$, còn $i + 1$ có cung tới A_i . Do đó, ta có thể dùng tìm kiếm nhị phân để tìm A_k sao cho $i + 1$

³Tức là một đồ thị có hướng gồm N đỉnh, trong đó giữa mỗi cặp đỉnh có đúng một cung theo một trong hai chiều.

có thể được chèn giữa A_k và A_{k+1} .

Giả sử ta đã biết $i + 1$ có thể được chèn giữa A_p và A_r . Ta hỏi chiều cung giữa $i + 1$ và $A_{(p+r)/2}$. Nếu $i + 1$ có cung tới $A_{(p+r)/2}$, điều đó cho biết $i + 1$ có thể được chèn trong đoạn từ A_p đến $A_{(p+r)/2}$. Ngược lại, nó có thể được chèn trong đoạn từ $A_{(p+r)/2}$ đến A_r . Cứ nhị phân như vậy cho đến khi $p + 1 = r$.

Nhờ đó, số câu hỏi cần dùng chỉ là $\mathcal{O}(n \log n)$.

Tiểu kết. Đối với dạng tìm kiếm nhị phân này, về bản chất ta đang tìm kiếm trên một dãy 01 vô thứ tự; đối tượng cần tìm là một xâu con đặc biệt “01”. Có xâu con này rồi, dùng phương pháp xây dựng, ta có thể chuyển kết quả thành một nghiệm khả thi của bài toán ban đầu.

Khi hỏi cung giữa $i + 1$ và $A_{(p+r)/2}$, nếu $i + 1$ có cung tới $A_{(p+r)/2}$, ta tìm cách chèn $i + 1$ vào giữa A_p và $A_{(p+r)/2}$. Thực ra điều đó không có nghĩa là $i + 1$ chắc chắn không thể chèn vào giữa $A_{(p+r)/2}$ và A_r ; chỉ là chưa chắc có thể. Còn $i + 1$ nhất định có thể chèn vào giữa A_p và $A_{(p+r)/2}$. Từ đó có thể thấy, phương pháp tìm kiếm nhị phân kiểu này thường áp dụng cho những bài toán có nhiều nghiệm khả thi, nhưng cần xây dựng một nghiệm khả thi một cách hiệu quả.

4.2 Mạch cổng NAND

Mô tả bài toán. Bài này đến từ ACM/ICPC CERC 2001. Có một loại cổng mạch gọi là cổng NAND. Mỗi cổng NAND có hai đầu vào, đầu ra là kết quả của phép NAND trên hai đầu vào, tức $\text{not}(A \text{ and } B)$. Cho một mạch không chu trình gồm N cổng NAND; tính không chu trình rất quan trọng đối với một mạch logic. Tất cả M đầu vào của mạch đều nối với cùng một đầu vào x , như sơ đồ NAND-1. Hãy đặt một số đầu vào thành hằng số, dùng số lượng x ít nhất để hoàn thành cùng chức năng. NAND-2 là một mạch chỉ dùng một đầu vào x nhưng có thể thu được cùng kết quả.

Phân tích. Ý tưởng đầu tiên là dùng biểu thức: biểu diễn đầu ra của mỗi cổng mạch thành kết quả của một phép tính hàm nào đó theo đầu vào, từ đó nối đầu vào và đầu ra của toàn mạch. Nhưng phép NAND không thỏa tính kết hợp, nên không thể tính toán biểu thức như vậy. Ta đành tìm một con đường khác.

Thực ra, ta rất dễ tính được đầu ra của mạch khi x lần lượt bằng 0 và 1, ký hiệu là ans0 và ans1 . Nếu $\text{ans0} = \text{ans1}$, rõ ràng ta không cần dùng đầu vào x nào; chỉ cần đặt toàn bộ đầu vào thành 0 hoặc 1 là thỏa yêu cầu. Nếu $\text{ans0} \neq \text{ans1}$ thì sao? Không dùng đầu vào nào là điều không thể. Vậy chỉ dùng một đầu vào thì sao?

Xét rằng, ngay cả khi chỉ có một x , số dãy đầu vào ta có thể chọn vẫn rất lớn; chỉ cần có một dãy thỏa yêu cầu là đủ. Vì vậy ta có thể mạnh dạn phỏng đoán rằng nhiều nhất chỉ cần một đầu vào chưa biết x là có thể thỏa yêu cầu.

Chỉ dùng một x nghĩa là khi $x = 0$, đầu ra của mạch đúng bằng ans0 ; khi $x = 1$, đầu ra của mạch đúng bằng ans1 . Ta chỉ thay đổi giá trị của một cổng đầu vào, và thay từ 0 thành 1.

Từ đó, ta tưởng tượng một dãy đầu vào toàn 0. Sau một số lần sửa đổi như trên, cuối cùng nó sẽ trở thành một dãy đầu vào toàn 1. Mặt khác, lúc bắt đầu đầu ra của mạch là ans0 , còn cuối cùng là ans1 . Vì vậy, chắc chắn tồn tại một thời điểm mà khi một đầu vào đổi từ 0 thành 1, đầu ra vừa đúng đổi từ ans0 sang ans1 .

Điều này chứng minh phỏng đoán vừa nêu: nhiều nhất chỉ cần một x . Đồng thời, thuật toán cũng

hiện ra. Giả tưởng rằng ta xây dựng $M + 1$ bộ đầu vào khác nhau như sau:

$$\begin{array}{lcl} 0 : & 0 & 0 & 0 & 0 & 0 & \dots & 0, \\ 1 : & 1 & 0 & 0 & 0 & 0 & \dots & 0, \\ 2 : & 1 & 1 & 0 & 0 & 0 & \dots & 0, \\ \vdots & & & & & & & \\ M : & 1 & 1 & 1 & 1 & 1 & \dots & 1. \end{array}$$

Ta cần tìm hai bộ đầu vào kề nhau sao cho bộ trước cho đầu ra $ans0$, còn bộ sau cho đầu ra $ans1$.

Việc này vẫn có thể hoàn thành hiệu quả bằng tìm kiếm nhị phân. Biết rằng bộ thứ p cho đầu ra $ans0$ và bộ thứ r cho đầu ra $ans1$, ta tính đầu ra ans' của bộ thứ $(p + r)/2$. Nếu $ans' = ans0$ thì lấy $r = (p + r)/2$; ngược lại lấy $p = (p + r)/2$, cho đến khi $p + 1 = r$.

Như vậy bài toán được giải quyết gọn trong thời gian $\mathcal{O}(N \log M)$.

Tiểu kết. Có thể thấy, điểm khó của dạng tìm kiếm nhị phân này nằm ở xây dựng. Làm thế nào đưa một bài toán tưởng như hoàn toàn không liên quan vào phạm vi ứng dụng của tìm kiếm nhị phân, rồi dùng một thay đổi nhỏ từ 0 sang 1 để xây dựng một lời giải cho bài toán ban đầu? Phương pháp xây dựng không có khuôn mẫu hay quy luật thống nhất; chỉ có thể phân tích cụ thể từng bài. Điều ta có thể làm là tiếp xúc nhiều hơn, suy nghĩ nhiều hơn và tích lũy nhiều hơn trong quá trình luyện tập.

5 Tổng kết

Bài viết này chỉ giới thiệu ngắn gọn vài ví dụ; việc bao quát toàn bộ, thậm chí phần lớn, các ứng dụng của chiến lược nhị phân là rất khó. Điều tôi muốn chỉ ra là: dù tư tưởng nhị phân đơn giản, nội dung của nó vẫn rất phong phú. Ta không nên để tư tưởng nhị phân chỉ dừng lại ở phép tìm kiếm nhị phân cơ bản nhất trên mảng có thứ tự thông thường, mà nên mở rộng nó sang các ứng dụng rộng hơn.

Chiến lược nhị phân thường kết hợp với một số thuật toán khác, nhờ đó che giấu bản thân và thoát khỏi tầm nhìn của chúng ta. Điều này đòi hỏi ta phải có nền tảng cơ bản vững chắc, kinh nghiệm giải bài phong phú, những phỏng đoán táo bạo nhưng hợp lý, cùng tư duy sáng tạo linh hoạt, thì mới có thể “lấy bất biến ứng vạn biến”.

Tài liệu tham khảo

1. Wu Wenhua và Wang Jiande, *Phân tích và thiết kế chương trình cho các thuật toán thực dụng*.
2. Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi đấu tin học*.